

研究進捗報告

青木心路 2026年6月9日

1 基本情報

- **報告日:** 2026年6月9日
- **報告期間:** 2026年5月19日 ~ 2026年6月9日
- **研究テーマ:** NMPCを用いたヘキサコプタの耐故障制御
- **作成者:** 青木心路

2 概要

卒業論文の研究テーマとして、NMPC（Nonlinear Model Predictive Control）を用いたヘキサコプタの耐故障制御の開発を行う。2026年9月～10月を目処に完成させることを目標とする。

3 現在の研究方針

- **使用マイコン:** Raspberry Pi Pico 2 W
- **制御周期:** NMPCを100Hzで動作させる（プラント積分は1kHz）
- **制御対象:** Hexa-X配置ヘキサコプタ（6モータ）の姿勢制御+高度PID
- **耐故障制御の方針:** NMPCの制約条件にモーター故障（ η ）を組み込み、加算合成制御（ $u_{cmd} = u_{baseline} + \Delta u$ ）で高度PIDと統合する
- **目標性能:** モーターが1～2個停止した場合でも、姿勢を水平に保ち、高度を一定に維持する
- **検証方針:** Pythonオフラインシミュレータ（`nmnpc_simulation.py`）で実機相当の非理想性を載せた上で性能評価し、その後Pico 2へ移植する

4 実施したこと

- **Python NMPCシミュレータ（`nmnpc_simulation.py`）の構築・拡張**
 - 6モータ・5状態（ ϕ, θ, p, q, r ）の凝縮形NMPC（ADMM + SR2/RTI）を実装
 - **意図:** Pico 2移植前にオフラインで制御性能を検証するため。`sample/nmnp.cpp`のPython鏡像とし、移植時に `Params` $\leftrightarrow$ `nmnpc_params.hpp` を突合しやすくする。ADMM + 凝縮形SR2 はリソース制約マイコン向けNMPC（NMPCM 先行研究）と同系の手法で、実機100Hz動作を前提とする
 - マルチレート構成：プラント1kHz（RK4）、NMPC 100Hz（ZOH）、予測ホライズン $N=30$
 - **意図:** 実機と同様に高速でプラントを積分しつつNMPCは100Hzで更新する構成を再現するため。 $N=30$ は旧 $N=20$ より予測を先読みし姿勢追従を改善する目的（計算負荷は増えるが100Hz目標に対し270～670Hzの余裕を確認済み）
 - 11次元プラントモデル（姿勢+高度+水平移動）を導入
 - **意図:** NMPCは姿勢5状態のみだが、姿勢制御の副作用として水平ドリフトが生じる。実機では高度・水平運動も同時に進むため、プラント側だけ11次元に拡張し、姿勢維持と並進の相互作用を評価できるようにする（NMPCコアは理想化モデルのまま据え置き）
- **制御統合パッケージ（PKG-A～F）の実装**
 - **PKG-A:** ascent $\rightarrow$ control 切替時の Bumpless Transfer（`w_ref/z_ref` ランプ、`Kd` フェードイン）

- 意図: 離陸は全機固定 PWM の open-loop 上昇、目標高度手前で高度 PID+NMPC に切替える。切替瞬間に $z_ref \cdot w_ref \cdot D$ 項がステップ入力となり推力が急変（バンプ）するのを防ぐ。 w_ref を切替時の上昇速度から 0 へランプし、 Kd を 0 から徐々に立ち上げ、 z_ref も線形ランプで接続する
- **PKG-B:** 高度PID再チューニング ($Kp=2.0$, $Kd=1.0$)
 - 意図: 旧ゲイン ($Kp=3.0$, $Kd=2.5$) では切替直後に $P \cdot D$ 項が大きく働き、推力が bang-bang 的に振動した。 Kp を下げて P キックを 33% 低減、 Kd を下げて $w=1$ m/s 時の D 項を機体推力の約 6% に抑え、滑らかなホバー遷移を実現する
- **PKG-C:** 加算合成制御 $u_cmd = u_baseline + \Delta u$ (NMPCは差分のみ最適化)
 - 意図: 高度制御を NMPC に入れると状態次元が増え計算負荷・設計が複雑化する（前回報告で懸念していた点）。Hexa-X では $B_c \cdot 1=0$ により平均モード（高度）と差分モード（姿勢）が分離するため、高度 PID が $u_baseline$ （ホバー推力配分）を担い、NMPC は姿勢補正の差分 Δu のみを最適化する構成を採用。ホバー時は $\Delta u \approx 0$ となり NMPC 負荷も軽い
- **PKG-D:** 故障時 Bumpless ($u_baseline$ ramp、PID $\cdot z_ref$ 再起動)
 - 意図: 故障で生存機への推力再配分が必要になると $u_baseline$ が急変し、制御入力にジャンプが入る。 $u_baseline$ を 0.3 s で ramp、PID 積分項を 50% 減衰、PKG-A と同様の $w_ref/z_ref/Kd$ ランプを再起動し、故障直後の姿勢・高度の乱れを抑える
- **PKG-F:** FDI未知時の制御層分離（制御層は $\eta=1$ 、プラントのみ真 η ）
 - 意図: FDI 推定器は未実装のため、制御器が故障を「知らない」状況を再現する。FDI 未知時は制御層は全機健全 ($\eta=1$) として推力配分・box 制約を設定し、プラントのみ真の η を適用。FDI 既知/未知の性能差 (S3a vs S3b) を定量比較し、推定器導入の必要性を評価するベースラインとする
- **案B:** 故障機ほど入力コスト R_u/η を大きくする $set_rbar_from_eta$
 - 意図: 部分故障機 (η が小さい) に NMPC が過剰な Δu を振り込むと、残存機の飽和や振動を招く。故障度が高いモータほど入力ペナルティを大きくし、生存機への姿勢補正を集中させる
- **実機リアリティ (PKG-E1~E8) の追加**
 - **PKG-E1** モータ一次遅れ ($\tau=50$ ms) ・ESC遅延 (2 ms)
 - 意図: 実機のモータ・ESC は PWM 指令に即応しない。遅れを無視したシミュ結果は実機で再現できないため
 - **PKG-E2** 物理パラメータ不一致（慣性 $\pm 20\%$ 、質量 $+5\%$ 、推力 -5% 等）
 - 意図: NMPC は公称パラメータで設計するが、実機は個体差・バッテリー重量等でずれる。モデル誤差下でのロバスト性を評価するため、プラント側のみ「真」パラメータをずらす
 - **PKG-E3** 空力推力モデル（プロペラ前進比効果）
 - 意図: 上昇中は誘導速度により推力が低下する。open-loop 離陸 PWM ($pwm_ascent=0.145$) の設計マージンにも反映
 - **PKG-E4** センサ遅延 (IMU 5 ms / 高度 20 ms)
 - 意図: 実機の IMU・気圧計には通信・フィルタ遅延がある。制御器が「古い」観測で動く状況を再現する
 - **PKG-E5** 水平移動 (x, y, u_v, v_v)
 - 意図: 姿勢傾斜による水平加速度・ドリフトを評価。外乱力 (S6/S7) の水平成分もプラントに反映する
 - **PKG-E6** プロペラジャイロ（簡易版）
 - 意図: 1 機停止で回転対称が崩れるとジャイロ効果が非ゼロになる。故障時の追加トルクを簡易モデル化
 - **PKG-E7** 地面効果（低空で最大 $+15\%$ 推力）
 - 意図: 離着陸付近では地面効果で推力が増える。低空フェーズの制御挙動をより実機に近づける

- **PKG-E8** ESC PWM 12-bit 量子化
 - **意図:** 実機 ESC は連続 duty ではなく離散 PWM。量子化による制御精度の限界を評価する
- **シナリオ評価 (S1~S8) の実施**
 - 健全機、部分/完全故障 (FDI既知・未知)、2機同時故障 (M5+M6)、外乱重畳など8パターン
 - **意図:** 耐故障制御の性能を体系的に比較する。S2a/b で FDI の影響、S3a/b で完全故障、S4 で 2 機同時故障 (旧 M3+M5 から M5+M6 に変更しヨー冗長性を残した現実的ケース)、S6~S8 で外乱重畳を検証
 - 起動時24項目のサニティチェックを追加
 - **意図:** PKG 追加・パラメータ変更のたびに回帰テストを自動化し、移植前に各機能が個別に正しく動くことを確認する
 - 詳細解説レポート (`nmnpc_simulation_report.md`) を作成
 - **意図:** 設計判断・パラメータ・実行結果を文書化し、Pico 2 移植時の仕様書兼レビュー資料とする
- **クアッド版シミュレータ (`nmnpc_simulation_quad.py`)** も同一PKG構成で整備 (NU=4、Quad-X配置)
 - **意図:** 4 発 1 機停止は 6 発より非線形乖離が大きく制御が難しい (前回報告の分析)。同一 PKG 構成でクアッド版を整備し、冗長性の違い (null space 1 次元 vs 3 次元) による耐故障性能の差を比較できるようにする
- 前回報告時点の方針どおり、まず6発機体での耐故障制御を優先する形でシミュレーション検証を進めた
 - **意図:** 6 発では NMPC 制約への故障組み込みみだけで耐故障制御が成立しやすく、飛行ロボコン機体も 6 発のため。まず 6 発で手法を確立し、実機移植の足がかりとする

5 分かったこと

- **6発機体はNMPC制約への故障組み込みみだけで耐故障制御が成立しやすい** (4発1機停止より非線形乖離が小さい)
- **加算合成 (PKG-C) + 高度PID** により、NMPCの次元増加なしに姿勢・高度を統合できる
- **FDI既知/未知で性能差が大きい**: S3a (既知) $\text{rms}_\phi \approx 0.05 \text{ rad}$ に対し、S3b (未知) $\text{rms}_\phi \approx 0.10 \text{ rad}$ 。未知時はNMPCが故障を補償せず、推力再配分は高度PIDのみが担う
- **N=30・コスト再調整 (Q_p/q増、Q_r低減)** で姿勢追従が大幅改善 (S1: $\text{rms}_\phi = 0.003 \text{ rad}$ 、旧N=20比で約7倍改善)
- **計算時間は100Hz目標に対し十分な余裕**: mean iter 1.5~3.4 ms → 実効270~670 Hz
- **M5+M6同時完全故障 (S4)** でも健全機並みの姿勢性能 ($\text{rms}_\phi \approx 0.004 \text{ rad}$) だが、一時的な入力飽和 (sat $\approx$ 26%) が発生
- Pico 2移植は N=30・H=180×180 で計算負荷が増えるため、パラメータ突合 (`Params` ↔ `nmnpc_params.hpp`) が次の課題

6 根拠

- NMPCM - Nonlinear Model Predictive Control on Resource-Constrained Microcontrollers.pdf
- Quasi-Newton Jacobian and Hessian Updates for Pseudospectral based NMPC.pdf
- `program/NMPC_simulation/nmpc_simulation_report.md` (2026-06-02版)
- 参照実装: `sample/nmpc.cpp`, `nmpc_model.cpp`, `nmpc_params.hpp`

7 実験・分析結果

Pythonオフラインシミュレーション (N=30, PKG-A~F適用、参照実行 img/20260602_144130/) の結果:

シナリオ	故障	FDI	rms_φ [rad]	rms_z [m]	sat [%]	mean iter [μs]
S1	なし	—	0.003	0.479	0.0	2962
S2a	M5 η=0.5	既知	0.031	0.489	0.4	3221
S2b	M5 η=0.5	未知	0.038	0.506	0.7	1563
S3a	M5 η=0	既知	0.046	0.536	13.1	3381
S3b	M5 η=0	未知	0.102	0.644	4.9	1543
S4	M5+M6 η=0	既知	0.004	0.482	26.2	1500
S6/S7	なし+外乱	—	0.003~0.012	0.479~0.485	0.0	1513~1652
S8	M5 η=0+τ_x外乱	既知	0.051	0.539	13.3	3291

- 健全機（S1）：姿勢RMS 0.003 rad、飽和0%、高度維持良好
- 部分故障（S2a/b）：FDI既知/未知とも rms_φ < 0.04 rad、案B + PKG-C が有効
- 完全故障1機（S3a）：制御可能だが sat≈13%
- 2機同時故障（S4）：姿勢は健全並みだが一時的飽和が大きい
- 実機実験は未実施。次段階でPico 2への移植・実機評価を行う予定

8 未解決事項・課題

- Pico 2への移植が未着手**：float64、H=180×180、N=30 で負荷増。Params と nmpc_params.hpp の突合が必要
- FDI推定器は未実装**：PKG-F で未知時の挙動は検証済みだが、推定→制御への反映は今後の課題
- NMPC内部モデルは公称値固定**：プラント側のみパラメータ不一致（PKG-E2）。モデル適応は未対応
- yaw (ψ) は未モデル化。水平移動は x,y のみ
- 試験用機体のCAD・実機配線、ESC基板の完成待ち
- pico_copter / Benzen へのNMPC移植作業は引き続き必要

9 次にすること

- Pythonシミュレータの制御ロジックを pico_copter / Benzen_NMPC へ移植し、Pico 2上で100Hz NMPCを動作させる
- Params と nmpc_params.hpp のパラメータ突合、計算時間・メモリ使用量の実機プロファイリング
- Raspberry Pi Pico 2 とジャイロセンサを接続し、姿勢制御の実機実験（健全機）を実施
- 実機で耐故障シナリオ（1機停止）の評価を行い、シミュレーション結果との比較
- 試験用機体の組み立て完了後、耐故障飛行試験へ移行